

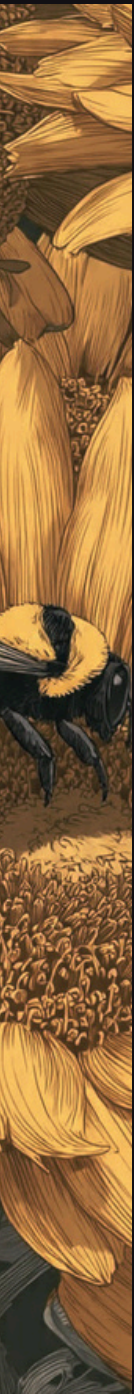
Past the AI Layer

FinOps for Agentic Development

Extending **FinOps** decision making to
early project cycles and **AI Assistance**

Spiresen

Louay Belkhamssa



Summary

FinOps for AI still manages cost the way traditional FinOps always has: after a system is deployed, through observability and correction. The Foundation's own 2026 Framework calls for shifting that work left, into design and architecture, but even its current "Architecting & Workload Placement" capability stops at the architecture of the AI system itself (orchestration, inference, RAG), not the architecture of the project that system is building. This deck argues for shifting further: treating the Software Engineering practice around it as cost-architecture decisions made before a single feature is written, using a real project, Spiresen Saga, as the working proof.



First with the basics

What is FinOps for AI?



FinOps is the framework and practices aimed at saving costs on the Cloud. Given that AI resources are majorly being hosted on the cloud, extending the framework to take AI into account is the natural next step towards mastering spend.

The framework itself is maintained and managed by the FinOps Foundation. Alongside it is Tokenomics, an organization also under the Linux Foundation but more focused on AI.

What's the status quo?

Traditionally, FinOps revolved around observability and governance. This meant that cost diagnosis and subsequent correction is post-mortem (after-the-fact). The same posture was applied to AI, favoring observability and **corrective actions** on systems that are **already deployed** (downstream)



Proactive upstream practices were rarely discussed for a couple of reasons:

- Tooling and practice are still lagging behind
- Saved costs cannot be reported if they never occur downstream thanks to good early decision making or architecture; this makes any future business case harder to sell

Understanding the trajectory

Is this enough?



Not quite. In the case of Agentic systems, costs are notoriously unpredictable, risking sudden spikes due to the probabilistic nature of LLMs, multiplied when working with multiple agents in large architectures.

A myriad of problems can occur, resulting in excessive token consumption:

- Infinite loops
- Context duplication
- Verbose output at scale

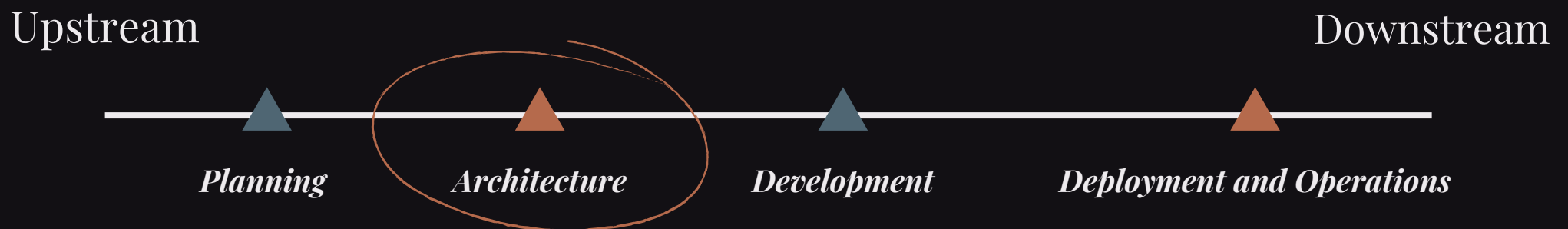
The result? **Budget allocation** for AI project and managing their **risks** has become increasingly **harder**.

Understanding the trajectory

What is shift-left?

Since 2025, the FinOps Foundation started pushing for more upstream practices figuring earlier on the project timeline, in other words: shift-left.

The solution is to extend the framework by putting in place **preventive measures upstream**, starting with architecture :



Your token spend is an **AI architecture** problem, not just a model problem

Arvind Jain, CEO of Glean

Architecture as means of cost management

There are plenty of handbooks and articles detailing the **standard practices** of what is now known as Tokenomics. The main elements eligible for optimization are the following:

Orchestration

How agents function and interact together within the same system

Inference

Token-based cost of running the model itself, input and output

RAG / Retrieval

Embeddings, vector storage, and inference cost inflated by injected context

Guardrails

Extra LLM calls spent filtering and evaluating outputs before they're trusted

Observability

logging and trace volume generated by the system watching itself

Training / Fine-tuning

GPU compute for adapting or hosting a model, distinct from using one

Taking a step back

A look on development phase of the work stream

While AI Assisted projects are on the rise, numbers are painting a **highly contrasted picture**.

AI use in development is increasing.

Cost for development is increasing and investments are pouring in.

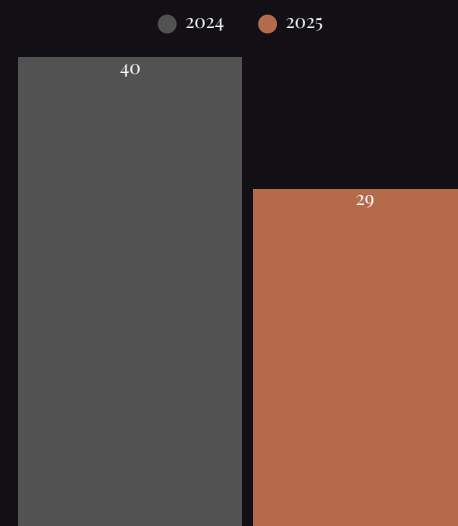
And yet, despite better models being released by the month, **trust** in what the AI is producing is diminishing year-on-year.

90%

Of profesional
devs using AI
weekly, 68% daily

+300%

Claude Code
business
subscriptions
since 2026-Q1



-11%

Trust of AI
output from
2024 to 2025

Analysing the issue

We identified 2 main problems:

- Current industry views on **AI assisted coding** revolve around the basics of optimisation (tool choice, budget spending, skills, etc.), with the **topic often raised after** the entire project has been agreed upon
- The quality of the output is a struggle to review and correct the code, leading to delays, **volatile** budget spending and **frustration** for the ones doing the work

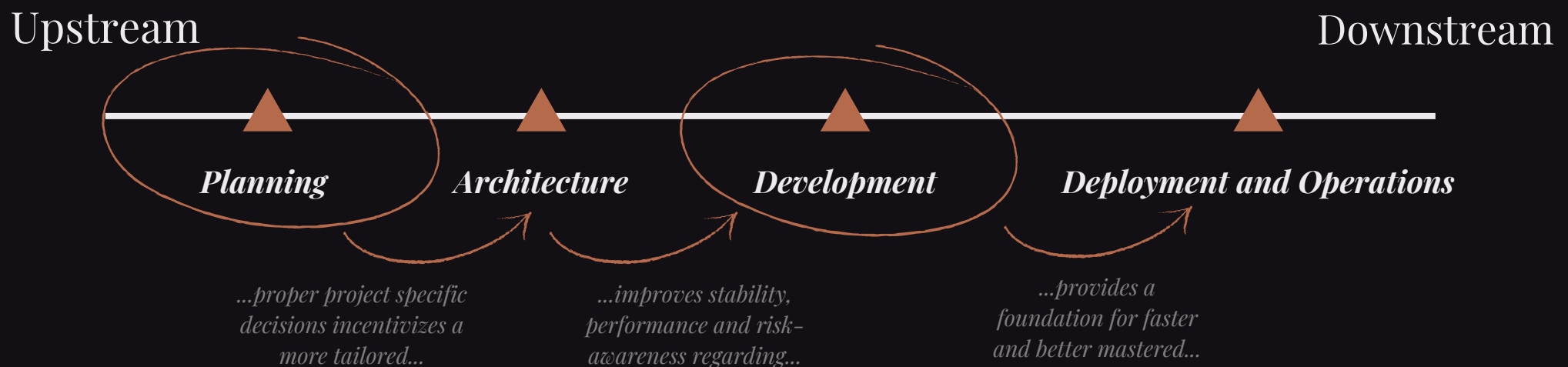
The question we're trying to answer becomes abundantly clear:



if we admit that AI is to be used within projects,

Why isn't AI Assistance rigorously factored into **the early planning** stages of the project, following proper **standards**?

Completing the loop



Beyond AI architecture

*Other dimensions to consider for AI Assisted coding
...and why*

Multiple research papers supporting our vision have already been published. Their goal is to identify technical development levers for reducing the AI Assistant's spend. A few examples of this would be:

Codebase and arch.

Stack used, level of documentation, libraries, code mapping, etc.

Clean Code

Code quality vs Assistant capabilities vs Spend tradeoff

Testing Strategy

Granularity of tests, Test-Driven Dev., AI code confidence beyond coverage

Branching strategy

How assistants can work and commit solo, ensuring tasks like review and checkups

In our work, we are trying to answer **practical, real-life questions**, moving beyond academic research:



Does the stack used influence my AI spend?

Yes, with every stack having its own **specificities**. Examples of issues being tackled are code mapping (discovery), verbosity, targeted editing of files, understanding of the code; each contributing to output quality and spend.

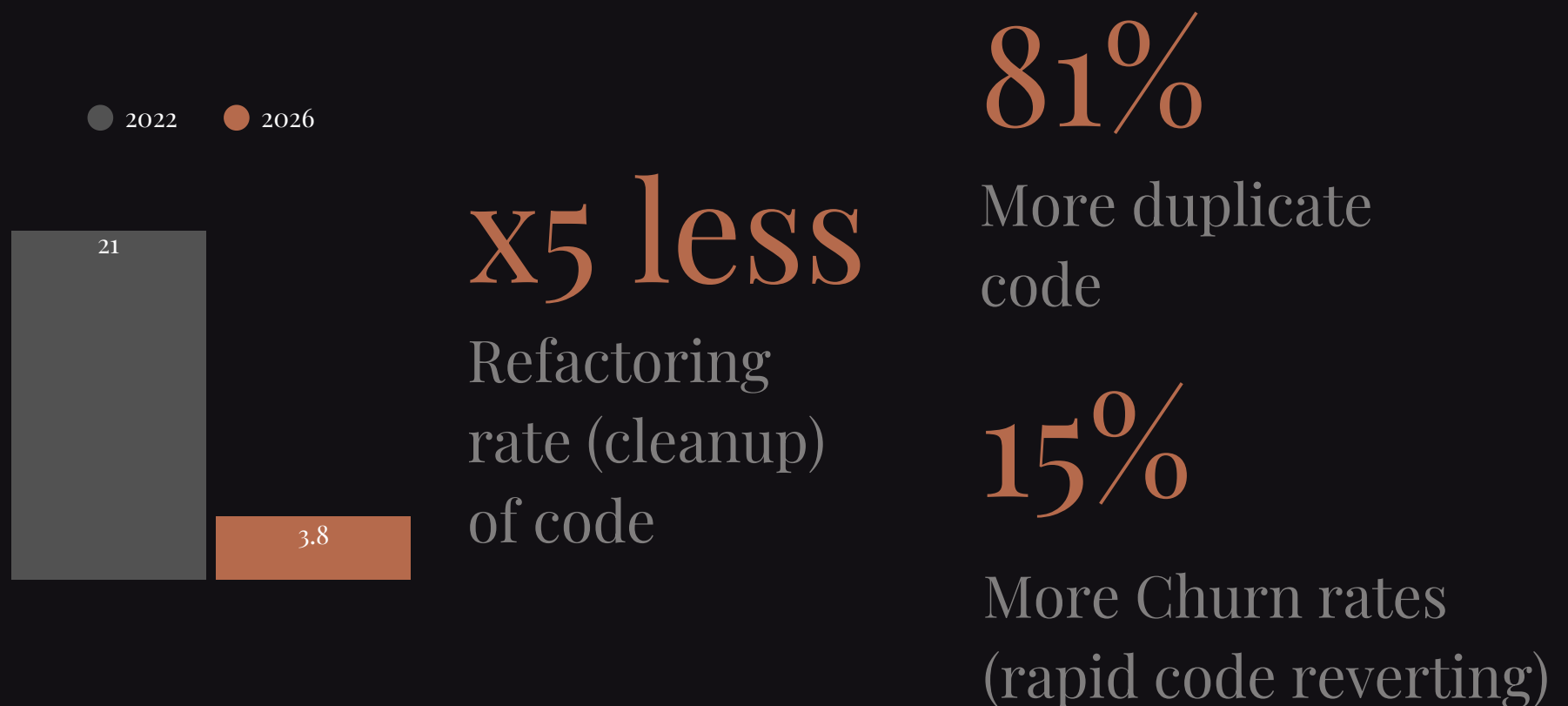
Should we, in a realistic setting, change the stack to save costs?

Not necessarily, and more often than not the answer is no. The stack is dependent on the team's familiarity with it and technical requirements of the product; all affecting quality, speed and performance.

That said, this doesn't mean taking the stack into account is futile. Admitting the tradeoffs and limitations of Code Assistants on a given project gives room for **proper handling of potential risks** with precise AI Assistant architecture corrections and more focused development practices.

AI tools cost nothing compared to the product and the engineers.

Deferred costs are hardly quantified, and are often shallow in their calculation. And while initial coding AI Assistance is merely fraction of the TCO, it plays a huge role in **long term maintenance**: delays, technical debt, and knowledge handover all play a role in increasing spend. Even worse, statistics show that the situation is **aggravating** for a lot of projects*:



**Not accounting for recent / future hikes of LLM fees (see Copilot June 2026)*

What's the end goal here?

Our contribution

This is an issue currently being studied by multiple teams, both privately and publicly. We do not claim we are at the origin of such thought, nor do we offer a one size fits all solution. However, the research is still granular and decentralized.

Our contribution would be an early suggestion of generalized FinOps standards and practices that could bring optimizations, quantified results and insights into this rather technical and specific section of the stream.



Our showcase project

Spiresen Saga

In our own little version of a project, we have gone over multiple topics so far, studying their impact (link at the end). While the scope isn't an enterprise-scale production app, **the decisions** and **their proofs** remain solid. Here are the elements we studied so far:

- Codebase architecture
- Clean Code Rules and tradeoffs
- Testing strategies
- Agentic Assistance workflows and architecture
 - Enforcement of researched decisions for coding, testing, deployment and maintenance.

For each of these, we have conducted research of existing **literature** to verify viability, created **criteria** for decision making and documented the verdict in **ADRs** (Architectural Decision Records).

Next target

A more in depth explanation of the project through a slide-deck is underway, but the initial documentation lives in the GitHub Repo.

The next step consists of showcasing **quantifiable gains** and **actionable insight** from each of these elements, proving their usefulness for a real project setting.

Risks

We also do acknowledge the existence of some risks, namely:

- Generalizability over most technical scenarios
- Agnosticism to the coding tool (e.g. Claude vs ChatGPT)
- Real-life project **constraints** limiting decision making freedom at the early stages of a project

And that's about it...

Thanks for reading!

Spiresen

Louay Bilkhamza

sources



Sources and relevant links:

Spiresen Saga - Github

Shift-left explanation - Harness.io

JetBrains Developer Ecosystem Survey 2026 - JetBrains

Stack Overflow 2025 Developer Survey - StackOverflow

Revenue growth announcement, Feb 2026 - Anthropic

The Maintainability Gap: AI Code Quality in 2026 - GitClear